

Day 11: Advanced PHP – OOP Basics & API Integration (6h)

Slide 1: What is OOP?

- **OOP = Object-Oriented Programming**
- (Objects) أسلوب برمجي يعتمد على الكائنات
- ...كل كائن يمثل كيان حقيقي مثل: طالب، سيارة، منتج

✓ الفوائد:

- **Modularity:** فصل كل جزء في كلاس مستقل
- **Reusability:** إعادة استخدام الكود بسهولة
- **Maintainability:** سهولة التعديل والتوسعة
- **Scalability:** مرونة في تطوير التطبيق مستقبلاً

✓ من OOP يتكون:

- كائنات (Objects)
- خصائص (Properties)
- دوال (Methods)

```
class Student {  
    public $name;  
    public function attend() {  
        echo "$this->name is attending...";  
    }  
}
```

Task: \ أنشئ كلاس `Book` يحتوي على خاصية `title` ودالة `read()` تطبع العنوان

Slide 2: Native vs OOP

Native PHP:

```
$name = "Ali";  
echo "$name is attending...";
```

OOP:

```
$student1 = new Student();  
$student1->name = "Ali";  
$student1->attend();
```

✓ الفائدة:

- كود أكثر نظمًا
- سهل إعادة استخدامه

Slide 3: Properties & Methods

```
class Student {  
    public $name;  
    public $email;  
  
    public function sayHello() {  
        echo "Hello $this->name";  
    }  
  
    public function showEmail() {  
        echo $this->email;  
    }  
}
```

Task: \ اصف `email` واستعملها بدالة `showEmail()`

Slide 4: Access Modifiers

Modifier	داخل الكلاس	خارج الكلاس	الوراثة
<code>public</code>	✓	✓	✓
<code>private</code>	✓	✗	✗
<code>protected</code>	✓	✗	✓

```
class User {  
    public $name = "Ali";  
    private $password = "12345";  
    protected $secret = "hidden";  
}
```

```

    public function getPassword() {
        return $this->password;
    }
}

```

Slide 5: Encapsulation vs Data Hiding

Concept	Description
Encapsulation	تجميع الخصائص والدوال داخل كائن
Data Hiding	إخفاء البيانات الحساسة باستخدام <code>private</code>

```

class BankAccount {
    private $balance = 0;

    public function deposit($amount) {
        $this->balance += $amount;
    }

    public function getBalance() {
        return $this->balance;
    }
}

```

Slide 6: Constructor

```

class Student {
    public $name;
    public $email;

    function __construct($name, $email) {
        $this->name = $name;
        $this->email = $email;
    }
}

$s = new Student("Ali", "ali@mail.com");
echo $s->name;

```

Slide 7: Inheritance

```
class User {
    public $name;
    public function login() {
        echo "$this->name logged in";
    }
}

class Student extends User {
    public $grade;
    public function study() {
        echo "$this->name is studying...";
    }
}
```

Slide 8: Traits

```
trait Logger {
    public function log($msg) {
        echo "[LOG] $msg";
    }
}

class Student {
    use Logger;
    public $name = "Ali";
}

$s = new Student();
$s->log("Student created");
```

Slide 9: مقارنة شاملة

Native:

```
$name = "Ali";
$email = "ali@mail.com";
echo "$name - $email";
```

OOP:

```
$student = new Student("Ali", "ali@mail.com");  
echo $student->name;
```

Slide 10: ربط OOP مع API

Native:

```
// post.php  
$name = $_POST['name'];  
echo json_encode(["name" => $name]);
```

OOP:

```
class Student {  
    public $name;  
    function __construct($name) {  
        $this->name = $name;  
    }  
    function toJson() {  
        return json_encode(["name" => $this->name]);  
    }  
}  
  
$s = new Student($_POST['name']);  
echo $s->toJson();
```

Slide 11: Task

- أنشئ كلاس Student
- name , email , age , private isActive
- أضف دالة activate() و getStatus()
- جرب استخدامه مع API POST

الخلاصة:

-  OOP vs Native

-  Properties, Methods, Access Control
 -  Inheritance, Traits
 -  Constructor, Encapsulation
 -  ربط API
-